

Inheriting a Team You Did Not Hire

The first ninety days with people who did not choose you

BY RASHAD MORGAN

Read enough IT leadership job descriptions and you notice a phrase that appears constantly and is almost never explained: *you will inherit existing teams*.

It shows up in postings for directors of technology operations, for integration leads, for anyone joining a company that has grown by acquisition. It is stated as context rather than as a challenge, as though inheriting a team were simply a starting condition rather than the hardest part of the first year.

It is the hardest part of the first year.

TWO WAYS THIS HAPPENS AND THEY ARE NOT THE SAME

You joined and the team came with the job. They have been operating without you, possibly for a long time. Someone among them may have applied for your role. They are wondering what you will change and whether they will like it.

Their company was acquired and you absorbed them. Everything above is true, plus they did not choose the acquiring company either, plus their employer changed without their consent, plus some of them are quietly wondering whether the acquirer needs two of every role.

The second is harder and the fear is more concrete. In the first case the question is whether they will enjoy working for you. In the second it is whether they will be working at all.

Most of what follows applies to both. Where it does not, I will say so.

THEY WERE COMPETENT BEFORE YOU ARRIVED

This is obvious and it is routinely ignored.

A team that has been running a service has knowledge you do not have: which system breaks on the last Friday of the month, which user needs to be called rather than emailed, which vendor actually answers the phone, why that server has not been patched and what happens if you patch it. None of this is written down. All of it is load-bearing.

New leaders arrive with a mandate to improve things and a reasonable instinct to demonstrate value early. The failure mode is changing something before understanding why it was that way. Sometimes the answer is that nobody ever fixed it. Often the answer is that it is that way for a reason that was not written down, and the person who knows the reason is sitting in the room watching you decide.

The practical version of this is simple. Spend the first weeks asking what works, not telling them what will change. Not as a technique for building rapport, though it does that. As a way of finding out what you do not know before you break something.

THE QUESTION THEY ACTUALLY HAVE

In an acquisition, every conversation you have in the first month is happening on top of an unasked question: am I keeping my job.

You will often not be able to answer it. Decisions may not be made, and you may not be the one making them. What you can do is be straight about the state of things.

There is an enormous difference between "I do not know yet" and silence. Silence gets filled with the worst available interpretation, and it gets filled fast. "I do not know yet, here is what I do know, here is when I expect to know more, and I will tell you when I do" is not a satisfying answer, but it is an honest one, and it is a promise you can keep.

Then keep it. Including when the update is that there is no update. The first time you say you will come back by Friday and do not, the credibility is gone and you will not get it back cheaply.

And do not promise what you cannot control. The temptation to reassure is strong and it is the single most damaging thing you can do. A leader who said everyone was safe and was then overruled has done more harm than one who said nothing.

LEARN THE ENVIRONMENT FROM THEM

Every acquired environment has undocumented dependencies. You will not find them in the inventory. You will find them by asking the people who have been operating it.

Useful questions, in roughly this order:

- What breaks most often, and what do you do about it
- What would you fix if someone gave you a month
- What is held together with something you would not want to explain
- Who actually knows how that works
- What have you tried to change before and what happened

That last one matters more than it looks. If a team has proposed improvements that went nowhere, they will not propose more to you until you give them a reason to believe it is different. And the history of what was tried tells you which constraints are real and which are just old decisions nobody revisited.

DECIDING WHO STAYS

At some point, in an acquisition, this becomes real. Two organizations have overlapping roles and the combined entity does not need all of them.

I am not going to pretend there is a way to do this that feels good. There is a way to do it that is defensible, and there is a way that damages everyone who remains.

What matters:

Decide on evidence, not on tenure with the acquirer. The instinct to protect your own people is strong and it is often wrong. The acquired company's technicians know an environment you do not. Judging them against a standard they have not had time to learn is a decision dressed up as a meritocracy.

Give people a real chance to be assessed. Ninety days of working under new expectations, clearly stated, is a fair basis. Two weeks of watching someone operate under the worst stress of their career is not.

Be fast once you know. Prolonged uncertainty is worse than bad news. People who suspect they are being managed out and are not told will leave on their own, and the ones with the most options leave first.

Say it yourself. If you made the decision, deliver it. Delegating that conversation to HR tells everyone watching what kind of leader they have.

The people who remain are watching how you treat the people who do not. That is not a reason to keep someone who should not stay. It is a reason to be straight, fast, and generous with the exit.

CHANGE THE THINGS THAT NEED CHANGING, ON A STATED TIMELINE

None of this means avoiding change. You were hired to change things, and a leader who spends a year listening and changes nothing has failed differently.

What helps is separating the two categories out loud.

Things changing soon, and why. Security standards, access controls, anything with a risk or compliance driver. These are usually not negotiable and it is better to say so plainly than to pretend there is a consultation when there is not.

Things changing later, with input. Tooling, process, how work is prioritized. This is where you genuinely want their view, and where you will get better answers than you would have reached alone.

Things not changing. Say this part. A team bracing for everything to be replaced will be relieved to hear that something they built is staying, and it costs you nothing to tell them.

The failure is not moving too fast or too slow. It is leaving people unable to predict which is coming.

WHAT YOU ARE ACTUALLY BUILDING

The point of doing this well is not that the transition feels better, though it does.

An inherited team that stays is the fastest route to understanding an environment you did not build. They know things that are not written down, and every person who leaves takes some of it with them. The retention conversation and the risk conversation are the same conversation.

And in a company that grows by acquisition, this is not a one-time event. You will do it again. How the first one went determines how much benefit of the doubt you get for the second, because people talk, and the acquired staff of the next company will hear about it.

Rashad Morgan leads shared services IT for a multi-brand industrial distribution group, where a merger and three acquisitions taught him most of what is in this piece. His part of the work is identity, service management, and end user support.

[More writing](#)